

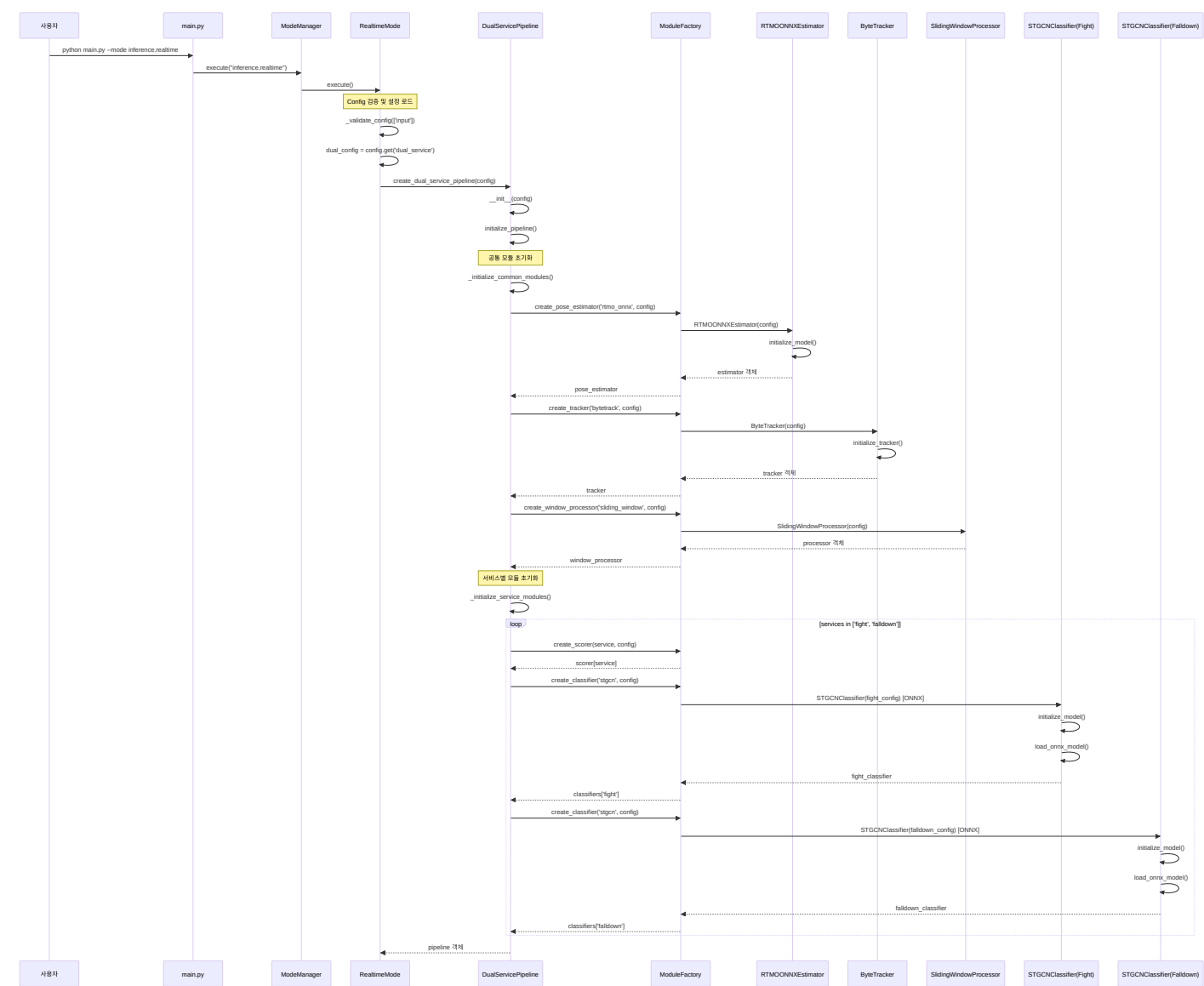
Inference.Realtime 모드 시퀀스 다이어그램

개요

이 문서는 inference.realtime 모드의 상세한 시퀀스 다이어그램을 ONNX와 PyTorch 적용 케이스로 구분하여 제공합니다.

ONNX 적용 시 시퀀스 다이어그램

1. 시스템 초기화 및 파이프라인 설정

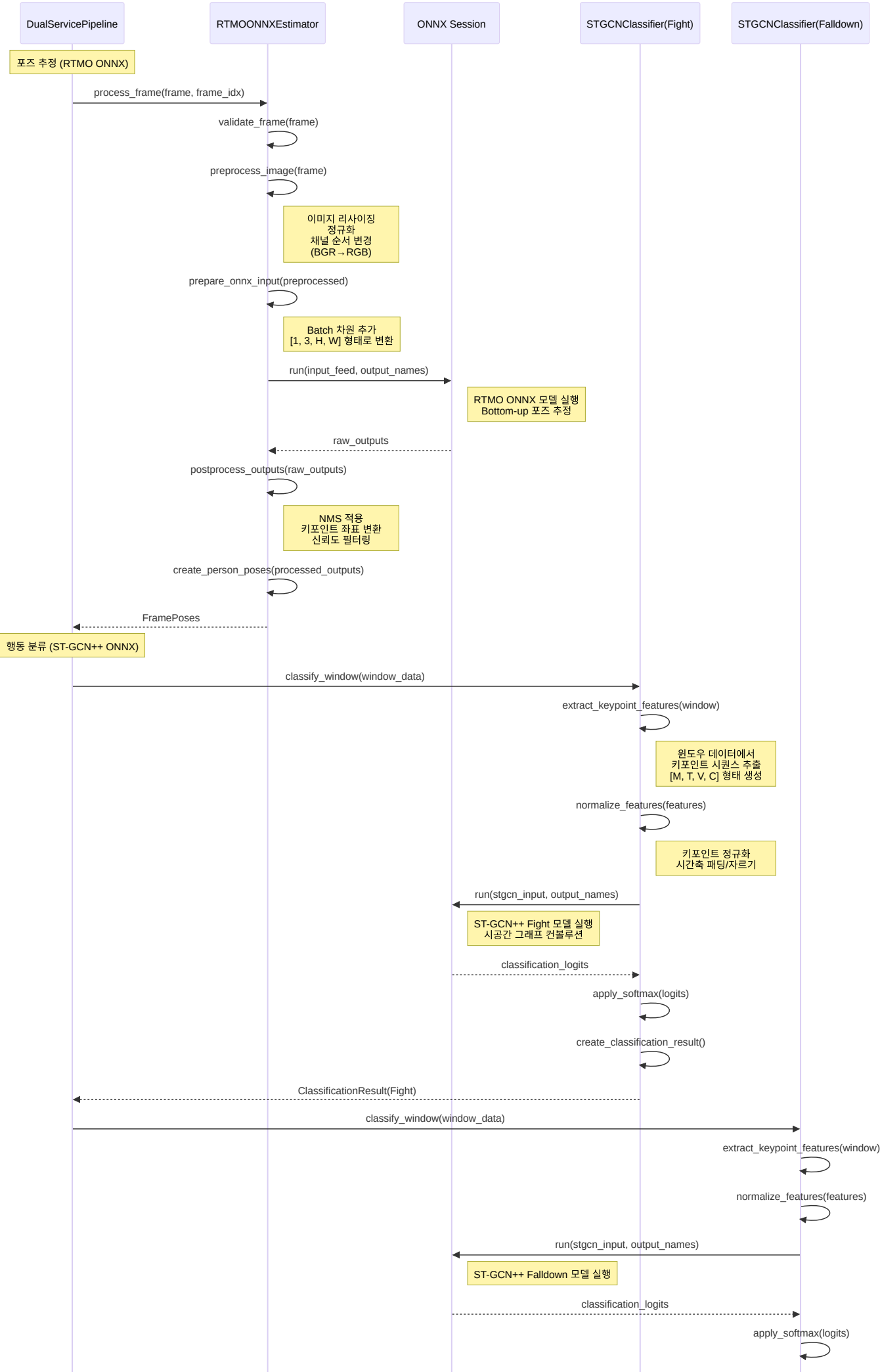


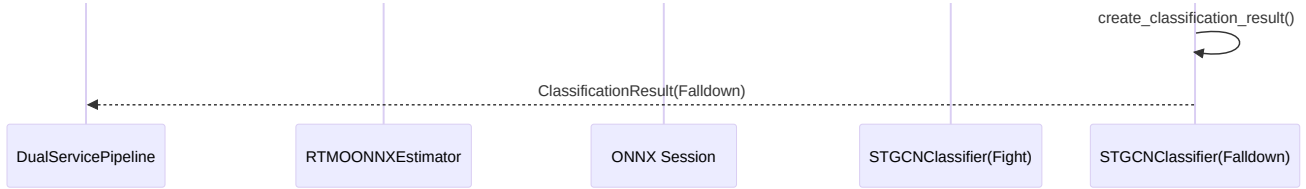
2. 실시간 처리 루프 (ONNX 기반)



Syntax error in text
mermaid version 11.9.0

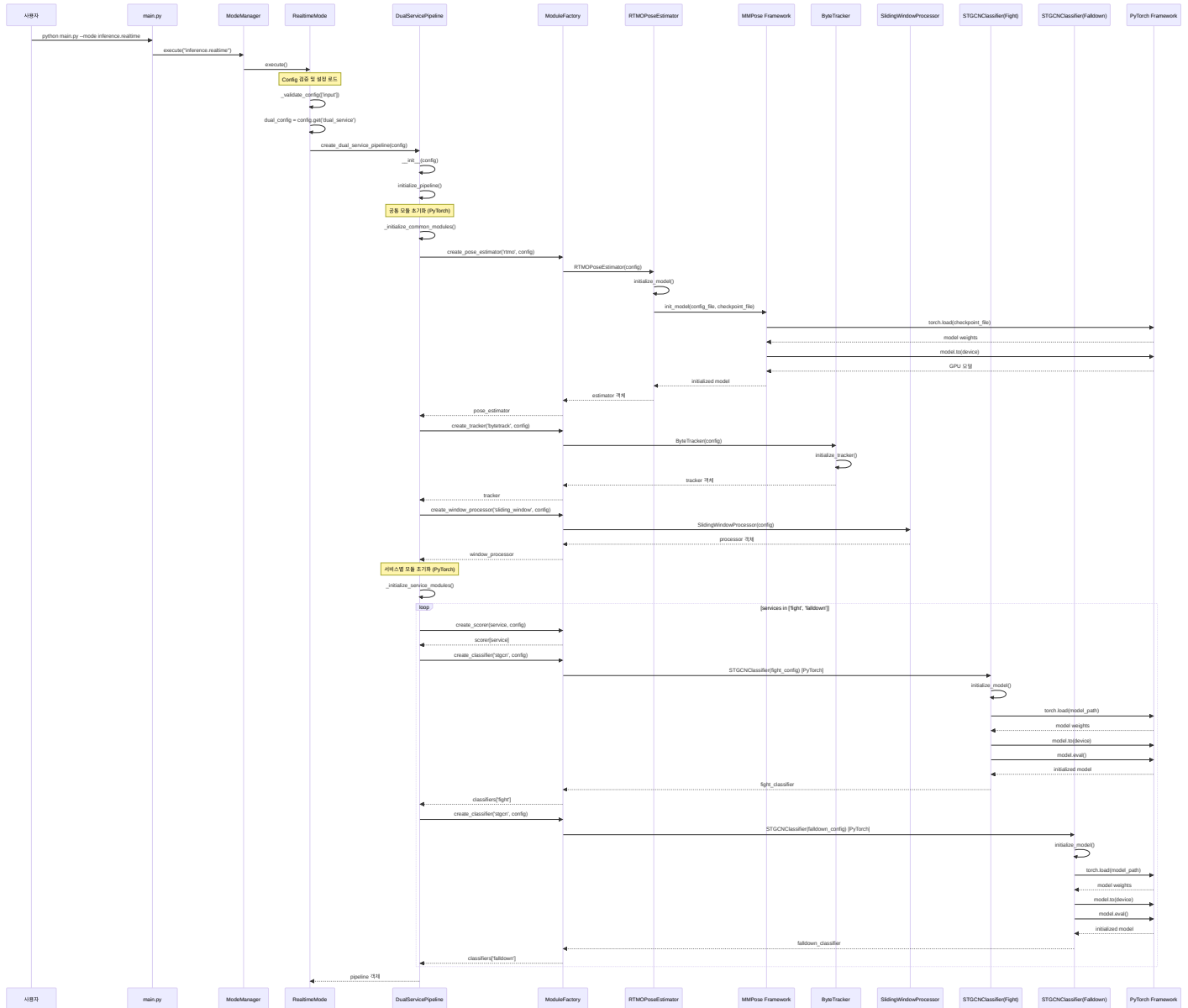
3. ONNX 모델 추론 상세 과정





PyTorch 적용 시 시퀀스 다이어그램

1. 시스템 초기화 및 파이프라인 설정 (PyTorch)

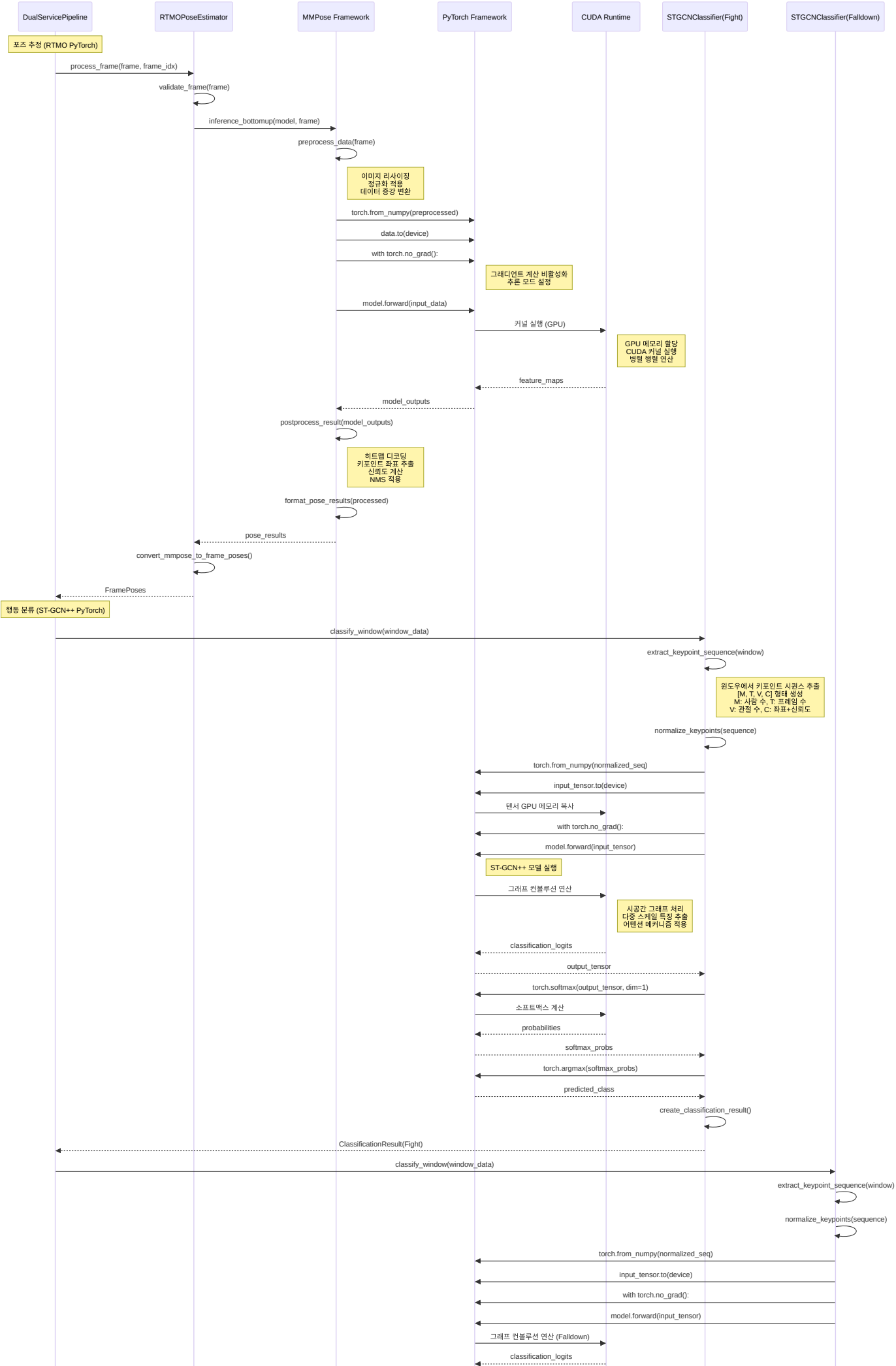


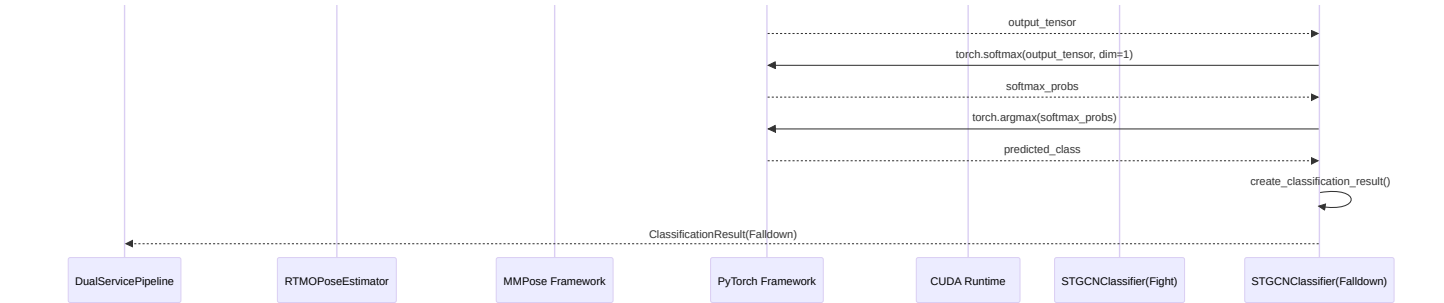
2. 실시간 처리 루프 (PyTorch 기반)



Syntax error in text
mermaid version 11.9.0

3. PyTorch 모델 추론 상세 과정





ONNX vs PyTorch 비교 분석

성능 특성 비교

구분	ONNX	PyTorch
초기화 시간	빠름 (모델 로딩만)	느림 (프레임워크 초기화 + 모델 로딩)
메모리 사용량	낮음 (최적화된 런타임)	높음 (프레임워크 오버헤드)
추론 속도	빠름 (최적화된 연산)	중간 (동적 그래프 오버헤드)
GPU 활용	최적화됨	우수 (네이티브 CUDA 지원)
배포 용이성	매우 좋음	보통 (의존성 많음)

기술적 차이점

ONNX 기반 처리

- 정적 그래프: 컴파일 타임에 그래프 최적화
- 전용 런타임: ONNX Runtime을 통한 최적화된 실행
- 플랫폼 독립성: 다양한 하드웨어에서 일관된 성능
- 메모리 효율성: 최소한의 메모리 오버헤드

PyTorch 기반 처리

- 동적 그래프: 런타임에 그래프 구성 및 실행
- 프레임워크 통합: MMPose와의 네이티브 통합
- 디버깅 용이성: 풍부한 디버깅 도구와 프로파일링
- 확장성: 새로운 모델과 기능의 쉬운 통합

사용 시나리오 권장사항

ONNX 권장 상황

- 프로덕션 환경 배포
- 제한된 하드웨어 리소스
- 높은 처리량이 필요한 경우
- 크로스 플랫폼 호환성이 중요한 경우

PyTorch 권장 상황

- 개발 및 실험 단계
- 모델 커스터마이징이 빈번한 경우
- 풍부한 디버깅 정보가 필요한 경우

- 새로운 기능 개발 및 테스트

결론

inference.realtime 모드는 ONNX와 PyTorch 두 가지 백엔드를 지원하여 다양한 배포 요구사항을 충족할 수 있습니다. ONNX는 프로덕션 환경에서의 성능과 효율성에 중점을 두고, PyTorch는 개발 단계에서의 유연성과 확장성을 제공합니다. 시스템의 전체적인 아키텍처는 동일하지만, 각 백엔드의 특성에 따라 모델 로딩, 추론 과정, 메모리 관리 방식에서 차이를 보입니다.